

Assignment # 4

Page No.	
Date	11/11/2023

Section-A

Q.1 What is demand paging?

⇒ Demand paging is a memory management scheme used in operating systems. It involves loading data into memory only when it is specifically requested, or "demanded", by a program during its execution. This approach allows for more efficient use of memory resources, as only the necessary portions of a program or data are brought into memory when needed, rather than loading the entire program at once.

Q.2 Explain memory management in UNIX.

⇒ In UNIX, memory management involves allocating and deallocating memory of for processes. The kernel manages this through mechanisms like virtual memory, utilizing RAM and swap space to optimize performance and ensure efficient use of resources.

Q.3 What is multilevel paging?

⇒ Multilevel paging is a memory management scheme used in computer operating systems. Instead of having a single page table to map virtual addresses to physical addresses, multilevel paging uses a hierarchy of page tables. This helps manage large address spaces more efficiently. Multilevel paging is beneficial for systems with

Teacher's Signature

Large address space, as it enables efficient memory management without the need to store an excessively large and sparse page table.

Q.4 Explain various swapping schemes.

1. Whole process Swapping :- Swaps out entire processes between main memory and secondary storage.

2. Partial Swapping :- Swaps only a portion of a process allowing more flexibility in memory management.

3. Page System :- Divides both the process and physical memory into fixed-size pages. Enables more efficient use of available memory.

4. Segmentation :- Divides both processes into logical segments. Each segment can be swapped independently, providing more flexibility in memory allocation.

5. Priority based Swapping

6. Demand paging

7. Contiguous Memory allocation :-

8. Non - Contiguous memory Allocation

Q.5 What is memory management?

⇒ Memory management is a crucial aspect of computer systems that involves controlling and organizing a computer's memory resources. It

Teacher's Signature

includes processes such as allocation and deallocation of memory space, ensuring efficient utilization of memory, and preventing conflicts or errors related to memory access. Memory management plays a vital role in optimizing a computer's performance and stability.

Section-B

Q.1 Cause and resolution for page interrupt faults, their impact, memory management, OS strategies, and system stability.

=> Page interrupt faults, often associated with virtual memory systems, can have various causes and resolutions. These faults occur when a program attempts to access a page that is not currently in physical memory. Here are key points:-

1. Causes:-

- **Page not in memory:** When a program references a page not present in physical memory.
- **Page protection violation:** Unauthorized access to a protected page.
- **Page fault during page replacement:** Occurs when a page is selected for replacement but is still in use.

2. Resolutions:-

- **Page replacement algorithms:-** Implementation like

LRU (Least Recently used or FIFO (First-In-FIRST OUT) determine which page to replace during a page fault.

- Page prefetching :- Proactively loading anticipated page into memory to reduce future page faults.
- Increasing physical memory :- Adding more RAM can decrease the frequency of page faults.

3. Impact :-

- Performance degradation :- Page faults incur a performance overhead due to the need to fetch data from slower storage.
- Increased I/O operations :- Page faults often trigger disk I/O operations to bring the required page into memory.
- System responsiveness :- Excessive page faults can lead to a less responsive system.

4. Memory management :-

- Paging and segmentation :- Techniques like paging and segmentation are employed for efficient memory management.
- Demand paging :- Pages are brought into memory only when needed, reducing unnecessary I/O.

5. OS strategies :-

- Page replacement policies :- OS decides which page to evict during a page fault based on specific algorithms.

- Write-back strategies:- Deciding when to write modified pages back to disk to optimize I/O.

6. System Stability:-

- Fault tolerance:- Implementing mechanisms to handle page faults gracefully without causing system crashes.

- Balancing memory demands:- Ensuring fair allocation of memory resources among competing processes.

Q-2 Role of virtual memory in efficient memory management, paging, segmentation, address translation and multitasking support.

⇒ Virtual memory plays a crucial role in efficient memory management by providing an illusion to the operating system and applications that there is more physical memory (RAM) available than there actually is. This is achieved through a combination of paging and segmentation.

1. Paging:-

- Virtual memory allows the operating system to divide the physical memory into fixed-size blocks called pages.

- Pages are used as the basic unit of data transfer between the disk and RAM.

- When a program needs more memory than is physically available, the operating system swaps data

between RAM and the storage device, loading only the necessary pages into RAM.

2. **Segmentation**:- Virtual memory supports segmentation, dividing a program into logical segments such as code, data, and stack.
 - Each segment is managed independently, allowing for flexible memory allocation and protection.
 - Segmentation helps in organizing and securing the memory space for different components of a program.

3. **Multitasking Support**:-

- Virtual memory enables multitasking by allowing multiple programs to run simultaneously, even if they collectively require more memory than the physical RAM can provide.
- Each program is allocated a portion of the virtual address space, and the operating system manages the swapping of pages in and out of RAM as needed.

Q.3 Impact and benefits of multi-level paging on memory management, address translation and system performance.

- ⇒ 1. **Reduced Memory Overhead**:- Multilevel paging helps in reducing memory overhead by breaking down the page tables into smaller manageable units. This is especially beneficial in systems with a large address space.

2. Improved Address Translation Efficiency:-
By organizing page tables in a hierarchical structure, multi-level paging minimizes the number of entries that need to be accessed during address translation. This hierarchical approach allows for faster traversal of the page tables, resulting in more efficient address translation.
3. Flexibility in Memory Management:-
Multi-level paging provides flexibility in memory allocation by allowing for dynamic adjustment of page table sizes based on the actual needs of the system. This can help in optimizing memory usage and accommodating varying application requirements.
4. Enhanced TLB (Translation Lookaside Buffer) Performance:-
The hierarchical structure of multi-level paging can lead to better utilization of TLB entries. This is because only the relevant portions of the page tables need to be loaded into the TLB, reducing the likelihood of TLB misses and improving translation speed.
5. Scalability:- Multi-level paging is highly scalable and can handle large address spaces efficiently. As the address space grows,

additional levels can be added to the page table hierarchy without requiring a complete redesign of the memory management system.

6. **Improved Page Fault Handling:-** The hierarchical structure allows for more granular control over page faults. Instead of loading entire page tables, only the necessary portions are brought into memory, reducing the impact of page faults on system performance.

7. **Enhanced Virtual Memory Support:-** Multi-level paging supports a larger virtual address space, facilitating the use of virtual memory techniques. This is crucial for modern operating systems and applications that require efficient handling of large datasets.

Section C

Q.1 Explain memory fragmentation, differentiating between internal and external fragmentation. Provide examples of how each type can occur and suggest strategies to mitigate their impact on system performance.

=> Memory fragmentation refers to the inefficient utilization of memory in a computer system. There are two main types :-

1. Internal Fragmentation:-

- **Definition:-** Occurs when memory is allocated in fixed-size blocks, and the allocated space may be larger than actually needed.
- **Example:-** If a process requires 12KB of memory and is allocated a fixed block of 16KB, 4KB is internally wasted.
- **Mitigation Strategy:-** Use dynamic memory allocation to allocate only the required memory, reducing internal fragmentation.

2. External Fragmentation:-

- **Definition:-** Occurs when free memory blocks are scattered throughout the system, making it challenging to allocate contiguous memory for larger processes.
- **Example:-** After multiple allocations and deallocations, there might be free memory scattered in smaller chunks, making it difficult to allocate a large contiguous block.
- **Mitigation Strategy:-** Techniques like compaction, where the free memory is consolidated into a single large block, can help reduce external fragmentation.

Q.2 Explain the concept of segmentation in memory management. Discuss its advantages, potential issues, and how it differs from paging. Provide examples to illustrate its practical use.

→ Segmentation in Memory Management:-

• Concept :-

- Divides a program into different segments (logical units) like code, data, stack.
- Each segment is allocated memory independently.
- Advantages :-
- Reflects the modular structure of a program.
- Supports dynamic data structures.
- Simplifies sharing and protection through access rights.

• Potential Issues :-

- Fragmentation can occur, leading to inefficient memory utilization.
- Varying segment size might complicate memory allocation algorithms.

• Differences from Paging :-

→ Unit of Allocation :-

- > Segmentation : Variable-sized logical units.
- > Paging : Fixed-sized blocks (pages)

→ Addressing :-

- > Segmentation : Logical addresses are divided into a segment number and offset.
- > Paging : Logical addresses directly mapped to physical addresses.

• Practical Use of Examples :-

→ Operating Systems :-

- > Segmentation used in many modern OS for

memory protection.

- Examples: Windows uses a combination of segmentation and paging.
- Database Management:-
- Segmentation helps in managing different parts of a database.
- For Instance, separating index information from actual data.
- Multitasking Environment:-
- Each task, can have its segments for code, data, and stack.
- Enhances isolation b/w tasks.

Q.3 Consider a page replacement algorithm with a reference string of length 20 and a fixed number of frames set to 3. Using the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1, analyze and compare the performance of the FIFO, LRU and optimal page replacement algorithms. Calculate the number of page faults for each algorithm and discuss their strength and weakness.

⇒ Let's analyze the performance of FIFO, LRU, and optimal page replacement algorithms with the given reference string "7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1" and three frames.

⇒ FIFO (First-In - First-Out):

1. Page faults: 15
2. Strength: Simple to implement
3. Weaknesses: Doesn't always perform well for all access patterns; may suffer from the Belady's anomaly.

⇒ LRU (Least Recently Used):-

1. Page faults: 12
2. Strengths: Tends to perform well in capturing temporal locality, considering recently used pages.
3. Weaknesses: Implementation can be more complex; may not always predict the best page to replace.

⇒ Optimal:-

1. Page faults: 9
2. Strengths: Provides the optimal page replacement strategy by replacing the page that will not be used for the longest time.
3. Weaknesses: Not practical to implement in real systems as it requires knowledge of future page accesses.